DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE

TM-40B

USAREUR-AF

CONCEPTS GUIDE — TM-40B COMPANION

AI ENGINEER

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

11 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

CONCEPTS GUIDE — TM-40B COMPANION

AI ENGINEER

MAVEN SMART SYSTEM (MSS)

HEADQUARTERS, UNITED STATES ARMY EUROPE AND AFRICA Wiesbaden, Germany 2026

PURPOSE: This guide develops the mental models required to design, build, and govern AI workflows on MSS effectively. It is a prerequisite companion to TM-40B and is intended to be read before beginning TM-40B task instruction.

DISTRIBUTION RESTRICTION: Approved for public release; distribution is unlimited.

TABLE OF CONTENTS

1. The AI Engineer's Role on MSS
2. How LLMs Actually Work — What You Must Understand
3. RAG and Grounding — The Operational AI Pattern
4. Decomposing an Operational AI Use Case
5. Agent Design Mental Model
6. Human-in-the-Loop as a Design Constraint, Not an Add-On
7. Prompt Engineering as Systems Engineering
8. MSS-Specific AI Engineering Mental Models
9. Common AI Engineering Failure Modes

DRAFT

SECTION 1 — THE AI ENGINEER'S ROLE ON MSS

BLUF: The AI engineer's job is to turn MSS data into AI-augmented workflows that increase decision velocity for operational users. This is not the same job as the ML engineer, the software engineer, or the ORSA — and conflating them produces poor design.

1-1. Role Distinctions on MSS

Three technical roles operate at the top of the MSS practitioner stack. Each has a distinct mission:

Role	Primary Output	Core Platform Tools	What They Are NOT Responsible For
AI Engineer (TM-40B)	AI-augmented workflows: LLM chains, Agents, grounded reasoning products	AIP Logic, Agent Studio, Code Workspaces	Model training, statistical inference, OSDK app architecture
ML Engineer (TM-40C)	Trained models, validation pipelines, model-backed Object properties	Code Workspaces, inference transforms, Foundry datasets	Prompt design, Agent orchestration, operational workflow UX
Software Engineer (TM-40F)	Custom applications, OSDK integrations, TypeScript UI components	OSDK, TypeScript SDK, Code Workspaces	LLM workflow design, model development, data pipeline authorship

These roles overlap at the edges and collaborate constantly. On a USAREUR-AF MSS project, the ML engineer builds a readiness prediction model; the AI engineer wraps that prediction in an AIP Logic workflow that synthesizes it with current LOGSTAT data and produces a commander's assessment draft; the software engineer surfaces that draft in a Workshop application with an approval widget. Each role has a lane.

The AI engineer's lane is the reasoning layer: the part of the system that takes structured data from the Ontology, routes it through an LLM, and produces a human-legible product for review.

1-2. What the AI Engineer Actually Produces

The AI engineer's primary deliverable is not code in the traditional sense — it is a **workflow**: a defined sequence of data retrieval, context assembly, LLM inference, and output handling that produces a useful product for an operational consumer.

On MSS, that workflow lives in AIP Logic or Agent Studio. It connects to the Foundry Ontology (the authoritative operational data model), retrieves Object properties, assembles a context window, calls an LLM inference endpoint, and delivers a structured output — a draft SITREP, a readiness summary, a

discrepancy flag, a recommended action for human review.

The measure of effectiveness for an AI workflow is not "does the LLM return something?" — it is "does this increase decision velocity without increasing decision error?" That distinction governs every design choice in TM-40B.

1-3. Where AI Engineering Starts and ML Engineering Ends

A useful boundary: **ML engineers teach the model; AI engineers use the model.**

The ML engineer is responsible for producing a model artifact — training it, evaluating it, versioning it, and deploying it via an inference transform to a Foundry dataset or as a computed Object property. The AI engineer is responsible for integrating that model output (or a general-purpose LLM endpoint) into an operational workflow that meets a defined use case.

In practice, this means:

- If a task involves selecting features, splitting datasets, tuning hyperparameters, or validating model performance metrics — that is ML engineering.
- If a task involves deciding what data to put in a prompt, how to structure a chain, what output format to require, and how a human reviews the result — that is AI engineering.
- If a task involves both (for example, a model-backed Object property feeding an AIP Logic workflow), the two engineers coordinate at the interface point. The AI engineer consumes model outputs; the ML engineer owns model quality.

1-4. AIP Logic, Agent Studio, and LLM Integration vs. Traditional ML

AIP Logic is a workflow authoring environment for LLM chains. You build a Logic workflow by defining: inputs (Object properties, user-provided text, retrieved data), a prompt template, inference parameters (model, temperature, output format), and output handlers (write to Object property, trigger Action, return to user).

Agent Studio is AIP Logic extended with memory, multi-step reasoning, and tool access. An Agent can call tools (search the Ontology, run a transform, retrieve a document), retain state across steps, and make sequential decisions. An Agent is appropriate when the task requires more than a single LLM call — when the answer depends on what the first query returns.

Traditional ML (classification, regression, forecasting) is not LLM-based. A readiness forecasting model, an equipment failure prediction model, a demand signal classifier — these are ML artifacts, not AI engineering artifacts. The AI engineer may consume their outputs, but does not build them.

DRAFT

SECTION 2 — HOW LLMS ACTUALLY WORK — WHAT YOU MUST UNDERSTAND

BLUF: You do not need to understand backpropagation to be an effective AI engineer. You do need to understand tokens, context windows, temperature, and the fundamental nature of LLM output. Gaps in this understanding produce unsafe workflows.

2-1. The Conceptual Minimum

A large language model is, at its core, a system that predicts the most probable next token given a sequence of preceding tokens. A token is roughly a word fragment — the word "readiness" is one token; the phrase "equipment readiness rate" is three. The model has learned statistical relationships between tokens from a massive corpus of text. When you submit a prompt, the model generates a response by sampling from the probability distribution of next tokens, one at a time, until it reaches a stop condition.

This has three non-obvious implications for operational AI engineering:

Implication 1: The same prompt can produce different outputs. Because the model samples from a probability distribution (not a lookup table), two identical prompts submitted at the same time can return different text. Temperature controls how broadly the model samples: high temperature = more randomness, lower temperature = more deterministic output. For operational workflows where consistency matters, use low temperature. Understand that even at temperature 0, some models are not fully deterministic.

Implication 2: The model is not a database. The model does not "know" the current readiness status of 1st Armored Division. It knows statistical patterns from its training data, which is both temporally bounded and domain-incomplete for military operational data. When you ask an LLM a factual question without grounding it in retrieved data, it will generate a plausible-sounding answer that is frequently wrong. This is not a bug — it is the fundamental nature of the architecture. The correct response to this fact is to ground every operational claim in data retrieved from MSS, not to hope the model knows.

Implication 3: More tokens in the context window does not always mean better output. LLMs have finite context windows — a maximum number of tokens they can process in a single call. As context length increases, models tend to "lose focus" on information positioned in the middle of the window. Relevance filtering before context assembly is not premature optimization — it is required practice.

2-2. Why LLMs "Hallucinate" and What That Means for Operational Use

"Hallucination" is the colloquial term for LLM output that is confidently stated but factually incorrect. It happens because the model's objective is to generate fluent, plausible text — not to verify facts. When the model lacks specific knowledge, it generates text consistent with patterns it has seen, which may be structurally correct but substantively wrong.

In an operational context, the risk is not that a hallucinated output will fool an expert analyst who knows the data. The risk is that it will be accepted by a consumer who does not know the data — a junior planner reviewing a pre-populated SITREP draft, a logistics officer seeing a generated demand forecast, an operations NCO reading an AI-generated equipment status summary.

The operational mitigation is not "build better prompts." Prompts reduce hallucination frequency; they do not eliminate it. The structural mitigation is grounding: every factual claim in an AI-generated product must be traceable to a specific retrieved data value from MSS. If a claim cannot be sourced, it should not appear in the output.

2-3. The Difference Between a Model Knowing Something and a Model Being Right

This distinction matters more than any other conceptual point in this section. LLMs do not flag uncertainty in the way a human expert would. A human analyst will say "I'm not sure about this, let me check." An LLM will generate text with the same syntactic confidence regardless of whether it is drawing on dense training data or extrapolating from thin analogy.

The practical implication: **never evaluate an LLM output based on how confident it sounds.** Evaluate it based on whether the factual claims are grounded in retrieved data you can verify. Any claim that is not grounded is a hypothesis, not a finding — and should be marked accordingly in the output format.

For USAREUR-AF workflows, this means: AI-generated products should distinguish, structurally, between retrieved facts (verifiable, source-cited) and synthesized assessments (LLM-generated, require human review). Merging these two categories into undifferentiated text is a workflow design failure.

SECTION 3 — RAG AND GROUNDING — THE OPERATIONAL AI PATTERN

BLUF: Retrieval-Augmented Generation (RAG) is the primary pattern for operational AI on MSS. Ground LLMs in Ontology data. Do not rely on model knowledge for operational facts.

3-1. What RAG Is

RAG is a workflow pattern: before calling the LLM, retrieve relevant data from a structured source and include it in the prompt context. The LLM then generates its response based on the retrieved data, not (or not primarily) on training knowledge.

On MSS, the retrieval source is the Foundry Ontology — the authoritative, continuously-updated model of USAREUR-AF operational data: unit readiness objects, equipment status, personnel accountability, logistics pipelines, exercise schedules, and more.

A simple RAG workflow for a readiness assessment: 1. Identify the unit Object(s) relevant to the user's query. 2. Retrieve the relevant Object properties: equipment readiness rates, personnel fill, maintenance backlog, supply status. 3. Assemble those properties into a structured context block. 4. Pass the context block and the user's question to the LLM with a prompt that instructs it to answer based on the provided data. 5. Return the response for human review.

Without step 2-3, the LLM would answer from training data — which contains no information about the unit's current status.

3-2. The Context Window as a Design Decision

The context window is not a dump zone. Every token you put in the context window costs inference time, costs money (if token-metered), and potentially degrades response quality by diluting relevance.

Context assembly is a design decision that requires the AI engineer to ask:

- **What does the LLM need to answer this question?** Not everything on the Object — the specific properties relevant to the query.
- **What format communicates the data most efficiently to the LLM?** Structured tables are generally more token-efficient than prose narratives for conveying multiple data points.
- **What can be filtered out?** Historical data not relevant to the current assessment, Object properties unrelated to the query, metadata fields the LLM will not use.

A common failure mode: assembling the entire Object record into context because it is easier than filtering. This is the AI engineering equivalent of briefing a commander on every LOGSTAT line item when they asked about critical shortfalls.

3-3. Context Breadth vs. Response Quality Tradeoff

Scenario	Context Breadth	Effect on Response
Narrow, targeted query	5-10 highly relevant properties	High precision, low hallucination risk, fast inference
Broad assessment query	30-50 properties, mixed relevance	More comprehensive but higher noise, increased hallucination risk, slower
Full record dump	Entire Object serialized	LLM may miss critical items buried in middle of context; no quality benefit over targeted

The correct engineering response to a broad assessment requirement is not to dump everything into context — it is to decompose the assessment into sub-queries, each with a targeted context assembly, and synthesize the sub-results in a final step. This is Agent territory (see Section 5).

3-4. Grounding as a Quality Guarantee

A workflow is "grounded" when every factual claim in its output can be directly traced to a retrieved value from the Ontology. Grounding is not aspirational — it is a structural property of the workflow that you either design in or leave out.

Grounding mechanisms: - Requiring the LLM to cite source Object and property for each factual claim (output format enforcement) - Post-processing the LLM output to verify cited values against retrieved data (validation transform) - Displaying retrieved source values alongside LLM-generated narrative in the user-facing product, so the reviewer can spot discrepancies

An ungrounded workflow produces a product that looks authoritative but cannot be verified. In a USAREUR-AF operational context, that product is a liability, not an asset.

SECTION 4 — DECOMPOSING AN OPERATIONAL AI USE CASE

BLUF: Before writing a single prompt, fully decompose the use case. What is the user doing? What data do they need? What does the LLM add? What is the human review step? Every use case gets all five questions answered in writing before development begins.

4-1. The Five-Question Framework

Every AI use case on MSS must be decomposed through this sequence:

Q1: What is the user trying to do? State the operational task in user terms — not in AI engineering terms. "The S4 needs to identify units with critical supply shortfalls before a major exercise" — not "we need an LLM to classify supply objects." The user task defines the measure of effectiveness.

Q2: What information do they need to do it? List the specific data elements the user requires to make the decision or produce the product. Be specific: "current supply status by class, by unit, for units assigned to exercise IRON RESOLVE, compared against the published 30-day sustainment standard." Vague requirements produce vague workflows.

Q3: Which of that information can be retrieved from MSS data? Map each required data element to a specific Ontology Object Type and property (or identify it as a gap). If a required data element does not exist in the Ontology, the workflow cannot provide it — and the user must be told that explicitly, not implicitly covered over by LLM-generated filler.

Q4: What does the LLM add on top of retrieved data? This is the AI value-add question. Common legitimate answers: synthesizes multiple data streams into narrative prose; identifies patterns across many Objects that a human would take hours to process manually; drafts a formatted product (SITREP,

status paragraph, discrepancy memo) from structured data; compares current status against doctrinal standard and flags deviations.

Illegitimate answers: "fills in information we don't have in the Ontology" (hallucination risk); "knows what the commander will want to see" (the model does not know this); "makes the assessment so the analyst doesn't have to" (unauthorized removal of human judgment).

Q5: What is the human review/approval step? This question is not optional. Document: Who reviews the output? What do they review it against? What action do they take to approve, reject, or modify it? What prevents the output from flowing into a downstream product or system without that review? This question gets answered in the workflow design before any other design decision is made.

4-2. Vignette: LOGSTAT Synthesis for Exercise Planning

A USAREUR-AF G4 planner needs to produce a pre-exercise logistics assessment for a brigade-level exercise. Current process: manually review LOGSTAT data across 8 subordinate battalions, compare against sustainment standards, draft a summary paragraph for the EXORD annex. Takes 4-6 hours.

Applying the five-question framework:

Question	Answer
Q1: What is the user trying to do?	Produce a logistics readiness assessment paragraph for EXORD annex
Q2: What information do they need?	Supply status by class (III, V, IX) by battalion; maintenance readiness; days-of-supply on hand vs. standard
Q3: What is in MSS?	Supply Objects with class/battalion/quantity properties; equipment Objects with maintenance status; MSS has all required data
Q4: What does LLM add?	Synthesizes 8 battalion data sets into a coherent prose paragraph; flags units below standard; formats to EXORD template
Q5: Human review step	G4 planner reviews draft paragraph against source LOGSTAT data; approves by clicking "Accept" in Workshop; output is held in draft state until approval is recorded

This decomposition reveals: the workflow is entirely feasible with full MSS grounding; the LLM's role is synthesis and formatting, not assessment; the human review step is the G4 planner, not an automated gate. Design begins from this specification.

DRAFT

SECTION 5 — AGENT DESIGN MENTAL MODEL

BLUF: An Agent is an LLM with access to tools and memory. Use Agents when a task requires sequential decisions based on intermediate results. Do not use Agents when a single LLM call is sufficient. Over-agentic design produces unpredictable, hard-to-audit workflows in operational contexts.

5-1. What an Agent Is — Conceptually

An Agent is a control loop: the LLM is given a task, decides what tool to call, calls it, receives the result, decides what to do next, and continues until it determines the task is complete or requires human input.

The tools available to an Agent on MSS include: Ontology search (retrieve Objects by type and property filter), property lookup (retrieve specific properties from a known Object), Action execution (write back to the Ontology — with appropriate governance gates), document retrieval (retrieve files from Foundry), and custom Python functions registered as tools.

The critical conceptual point: the Agent decides which tools to call and in what order. You, the AI engineer, do not control the exact execution path — you control the available tools, the system prompt, and the termination conditions. This is a fundamentally different design paradigm than a deterministic pipeline.

5-2. When to Use a Single LLM Call vs. a Multi-Step Agent

Use Case	Single Call	Agent
Format structured data as prose	Yes	No
Answer a question about known Objects	Yes (with RAG)	No
Synthesize a SITREP from a defined data set	Yes	No
Investigate an anomaly that requires sequential lookups	No	Yes
Complete a task where the next step depends on what the previous step returned	No	Yes
Automate a multi-step workflow with decision points	No	Yes

The default is a single LLM call. Upgrade to Agent only when the task genuinely requires it. Every Agent you build is more complex to test, audit, and explain to a commander than every single-call workflow you build.

5-3. Agent Workflows as Decision Trees with Uncertainty

A useful mental model: an Agent workflow is a decision tree, except you do not know in advance which branches will be taken — the LLM decides at runtime. This has implications:

- You cannot enumerate all possible execution paths for testing. You can test common paths and adversarial inputs, but the search space is not finite.
- The Agent can get stuck in loops. Without explicit loop-detection and hard step limits, an Agent that fails to find satisfying tool results may call the same tool repeatedly.
- The Agent's "reasoning" is not transparent. Unlike a deterministic pipeline where you can read the code and trace the execution, the Agent's step-by-step decisions emerge from LLM inference and are not introspectable in the traditional engineering sense.

Design implications: set hard step limits (maximum N tool calls before forced termination); log every tool call and result; define explicit termination conditions; test failure cases first.

5-4. The Failure Mode of Over-Agentic Design

Over-agentic design is the single most common AI engineering mistake in operational environments. It manifests as:

- Using an Agent for a task that could be accomplished with a single LLM call plus RAG
- Building Agents with access to more tools than the task requires
- Allowing an Agent to make write operations (Actions) without a human checkpoint at each write
- Treating Agent completion as task completion — without reviewing what tools were called and why
- Building Agent complexity because it is impressive in a demo, not because the task requires it

In a USAREUR-AF operational context, over-agentic design is a safety issue. An Agent with unnecessary tool access and no step logging is an AI system that can modify operational data in ways that are difficult to trace, explain, or reverse. The complexity that makes Agents powerful also makes them opaque — and opacity is incompatible with operational accountability.

The design heuristic: if you cannot explain to the commander what the Agent will do on every input type, the Agent is not ready for operational deployment.

SECTION 6 — HUMAN-IN-THE-LOOP AS A DESIGN CONSTRAINT, NOT AN ADD-ON

BLUF: In USAREUR-AF, human review before action is a doctrinal and legal requirement for AI-assisted operational decisions. Design the human review step first, then design the AI workflow around it.

DRAFT

6-1. The Doctrinal Requirement

Army CIO Policy (April 2024) requires human oversight of AI-assisted operational decisions. USAREUR-AF supplements this with C2DAO governance requirements for all AIP Logic workflows that process operational data or produce products used to inform command decisions. These are not suggestions — they are requirements that must be documented in the workflow design before deployment authorization is requested.

The requirement is not "a human sees the output." The requirement is: a qualified human with the information needed to evaluate the output reviews it and makes an affirmative decision to accept, reject, or modify it before it flows into an operational product or triggers an action.

6-2. Meaningful Review vs. Rubber-Stamping

The difference between meaningful human review and rubber-stamping is not the reviewer's rank — it is the reviewer's ability to evaluate what they are reviewing.

A human reviewer can meaningfully evaluate AI output when: - The output is clearly structured (assessments vs. retrieved facts are visually distinct) - The source data for retrieved facts is displayed alongside the LLM output - The reviewer has enough domain knowledge to evaluate whether the LLM assessment is plausible - The review interface provides explicit accept/reject/modify options, not just an OK button - The time allotted for review is sufficient to actually read and evaluate the output

A human reviewer is rubber-stamping when: - The output is a wall of LLM-generated prose with no source citations - The review interface is a single "Submit" button with no option to modify - The reviewer is under time pressure that makes careful evaluation impractical - The output is technically correct-sounding enough that a non-expert has no basis to question it - The workflow is designed to be reviewed in 30 seconds and the output requires 5 minutes to evaluate properly

Rubber-stamping is worse than no human review. It creates a false accountability record — a log that says "human approved" when no meaningful evaluation occurred. Design review interfaces that enable genuine evaluation.

DRAFT

6-3. Design Patterns for Meaningful Review

Design Pattern	What It Does	Why It Matters
Source citation in output	Every factual claim links to the Ontology Object/property it came from	Reviewer can verify claims against source in the same interface
Side-by-side display	Retrieved data displayed alongside LLM synthesis	Reviewer sees what the LLM saw; can spot discrepancies
Explicit assessment tagging	LLM assessments labeled as AI-generated	Reviewer knows what requires their judgment vs. what is factual retrieval
Modification field	Reviewer can edit the AI output before approving	Promotes active engagement; output reflects reviewer judgment
Audit log	Accept/reject/modify logged with reviewer identity and timestamp	Accountability record is meaningful, not just a click log
Step review for Agents	Each Agent action requires review before next step executes	Prevents Agent from compounding an early error through subsequent steps

6-4. The Approval Chain as a Workflow Design Element

For operational products, the human review step is not just a quality gate — it is part of the staffing process. A G4 planner reviews a logistics assessment draft; the G4 approves and forwards to the DCofS; the DCofS includes it in the EXORD package. That approval chain must be reflected in the Workshop application workflow, not bypassed.

AI engineering that treats the approval chain as a formality to be streamlined has misunderstood its operational context. The approval chain exists because the product will inform command decisions. Streamlining the chain means reducing the oversight of decisions — which is the opposite of the objective.

SECTION 7 — PROMPT ENGINEERING AS SYSTEMS ENGINEERING

BLUF: A prompt is system configuration. Treat it as such. Version-control it, test it against edge cases, document its design intent, and plan for the failure mode.

7-1. Prompts Are Not Casual Instructions

In a consumer AI context, prompts are conversational. In an operational AI engineering context, prompts are system configuration files that govern the behavior of a component in a live operational pipeline. The standards that apply to any other system configuration apply to prompts:

- **Version control.** Every prompt in production has a version, a change log, and a review history. You know what changed between v1.2 and v1.3 and why.
- **Testing.** Prompts are tested against representative input sets before deployment, including edge cases and adversarial inputs.
- **Documentation.** Every prompt has documented design intent: what it is designed to produce, what inputs it is designed to handle, and what it explicitly does not handle.
- **Change management.** Prompt changes go through the same review process as code changes. An untested prompt change is a code push without tests — unacceptable in production.

7-2. Prompt Anatomy for Operational Workflows

A well-structured operational prompt has four components:

Role definition. Establishes the LLM's operational context and constraints. Example: "You are a logistics analysis assistant supporting USAREUR-AF G4 planning. Your role is to synthesize supply status data into clear, concise assessment paragraphs. You do not make recommendations outside the data provided. You do not speculate about future conditions unless explicitly asked."

Task specification. States precisely what the LLM is to produce. Include output format, length, and structure requirements. Example: "Produce a two-paragraph logistics readiness assessment for the unit data provided below. Paragraph 1: current supply status by class, flagging any class below 80% on-hand. Paragraph 2: maintenance readiness summary, flagging any equipment category below 85% operational. Use the USAREUR-AF LOGSTAT paragraph format."

Data block. The retrieved Ontology data, formatted for LLM consumption. Structured tables are more token-efficient than serialized Object JSON. This block is dynamically populated at runtime — it is not static.

Constraint block. Explicit statements of what the LLM must not do. Example: "Do not include information not present in the data block. Do not make recommendations regarding personnel actions. If the data block is incomplete for any unit, flag the gap explicitly — do not estimate missing values."

7-3. Testing Prompts Against Edge Cases

A prompt that works 95% of the time is a production risk in an operational workflow. The 5% failure cases matter more, not less, in military operational contexts — because high-tempo, high-stakes conditions are precisely when data quality degrades and edge cases become common.

DRAFT

Required test categories for every operational prompt:

Test Category	What to Test	Why
Nominal inputs	Representative operational data within expected ranges	Establish baseline performance
Missing data	Data block with one or more required fields absent	LLM must flag gaps, not estimate
Boundary values	Data at threshold values (exactly 80% readiness, exactly 30 days supply)	Ensure thresholds produce correct flags
Contradictory data	Data block with internally inconsistent values	LLM must flag, not resolve arbitrarily
Adversarial inputs	Data block with values designed to elicit hallucination	Test grounding enforcement
Off-task inputs	User query that is outside the workflow's scope	LLM must decline gracefully, not improvise

Document test results. If a prompt fails a test category, fix the prompt and retest — do not accept a known failure mode and deploy anyway.

7-4. The Operational Risk of a 95% Prompt

Suppose a SITREP synthesis workflow produces accurate output 95% of the time. In a garrison environment processing 20 SITREPs per week, that is 1 error per week. In a high-tempo exercise environment processing 100 SITREPs in 72 hours, that is 5 errors — potentially during the most demanding period of the exercise.

The design response to this math: 1. Know your failure modes. If you have tested the prompt, you know which input types produce errors. Document them. 2. Surface uncertainty. When the workflow detects a condition that has historically produced errors (missing data, contradictory values, off-nominal inputs), flag it explicitly in the output rather than generating a normal-looking but unreliable product. 3. Grade the output. A well-designed workflow does not produce identical-looking outputs for high-confidence and low-confidence cases. The reviewer should be able to see, at a glance, whether this output requires cursory review or careful scrutiny.

SECTION 8 — MSS-SPECIFIC AI ENGINEERING MENTAL MODELS

BLUF: AIP Logic and Agent Studio operate within the Foundry Ontology. Understanding the Ontology relationship is not background knowledge — it is the foundation of every design decision.

8-1. AIP Logic and the Foundry Ontology

AIP Logic is not a standalone AI tool — it is an AI layer on top of the Foundry Ontology. This relationship has practical implications:

Inputs to AIP Logic workflows come from the Ontology. Object properties, filtered Object sets, linked Objects — these are the data sources. The quality of your AI workflow's output is bounded by the quality of the Ontology data feeding it. Garbage-in-garbage-out applies to LLM workflows exactly as it applies to dashboards and reports.

Outputs from AIP Logic workflows can write back to the Ontology. Through Actions, an AIP Logic workflow can update Object properties, create new Objects, or trigger downstream pipeline events. This is powerful and dangerous in equal measure. A misconfigured or malfunctioning workflow that writes to the Ontology does not just produce a bad output — it corrupts the authoritative data that every other MSS consumer relies on.

The Ontology model constrains what is possible. If the Ontology does not have a property for "AI assessment confidence level," you cannot write confidence metadata back to the Object — unless you add the property (which requires Ontology governance). Never hack around Ontology constraints by stuffing structured data into free-text fields. Coordinate the Ontology change through the correct channel.

8-2. Agent Memory in a Multi-User Operational Context

Agent memory — the ability for an Agent to retain context across interactions — is more complex in a multi-user operational environment than in a consumer application.

Mental model: Agent memory is a state variable. In a single-user consumer context, memory belongs to one user and accumulates personal context over time. In a multi-user operational context, memory design requires answering:

- **Whose memory is this?** User-level memory (specific to one analyst's session) or workflow-level memory (shared across all users of the workflow)?
- **What goes in memory?** Prior queries, retrieved data, interim assessments, user preferences — each category has different sensitivity and retention implications.
- **How long is memory retained?** Session-level (cleared after the session ends), exercise-level (cleared after the exercise concludes), persistent (retained indefinitely)?
- **Who can see what is in memory?** In an operational environment with role-based access control, memory content may need to respect the same access restrictions as the source data.

Failing to answer these questions before deploying an Agent in a multi-user environment can result in context from one user's session influencing another user's outputs — a subtle data spillage problem that is difficult to detect and diagnose.

8-3. C2DAO Review as the Quality Gate

C2DAO (Command and Control Data Analytics Office) review for AI workflows is not bureaucratic process — it is the governance gate that prevents an AI workflow from corrupting operational data or producing unauthorized AI-assisted decisions at scale.

The C2DAO review evaluates: - Does the workflow write back to the Ontology? If so, what does it write, to which Objects, under what conditions? - What is the human review step? Is it meaningful (see Section 6) or rubber-stamping? - Is the workflow grounded? Are all factual claims in outputs traceable to retrieved Ontology data? - What are the documented failure modes? Has the workflow been tested against them? - What is the rollback procedure if the workflow produces incorrect outputs at scale?

A workflow that cannot answer these questions clearly is not ready for C2DAO review. The AI engineer's job is to make the answers to these questions obvious from the workflow documentation, not to argue for an exception.

Think of C2DAO review as the pre-deployment checklist that every pilot completes, not as a speed bump. Skipping it is not faster — it is the failure mode.

8-4. Governance Before Activation — The Correct Sequence

The correct sequence for deploying an AIP Logic workflow to production:

1. Decompose the use case (Section 4) and document it.
2. Design the human review step (Section 6) and document it.
3. Build the workflow in a development environment against synthetic or development-tier Ontology data.
4. Test against the required test categories (Section 7-3).
5. Document failure modes, confidence levels, and known limitations.
6. Submit for C2DAO review with the workflow documentation.
7. Receive C2DAO authorization.
8. Deploy to production.
9. Monitor output quality post-deployment; report anomalies to C2DAO.

Any sequence that moves step 7 later is not a faster path to deployment — it is a path to deploying an ungoverned workflow, which is not authorized.

SECTION 9 — COMMON AI ENGINEERING FAILURE MODES

BLUF: Most AI engineering failures on operational platforms are not technical failures — they are design failures. They happen before a line of code is written. Recognize them early.

9-1. Over-Promising Capability

The most common failure mode at the project inception stage: scoping the AI workflow as if the LLM can do everything the user might want, without grounding the scope in what MSS data actually supports.

Common manifestations: - "This Agent can replace the S2 analyst" — No. It can accelerate specific S2 analytical tasks that are well-defined, data-complete, and appropriately scoped for AI-assisted analysis. It cannot replace professional military judgment, source evaluation, or analytical tradecraft. - "This workflow will automate the SITREP process" — No. It will draft SITREP paragraphs from retrieved data, which a human analyst will review and finalize. The workflow automates data retrieval and formatting; it does not automate the analytical judgment. - "This will save 8 hours per day for the operations section" — Maybe. State that as a hypothesis to be validated, not a committed outcome. Commit to what the workflow does; let the user's time savings be measured after deployment.

Over-promised capabilities create expectation gaps that erode trust in AI tooling when the delivered system meets its actual (correct) scope but not the inflated pre-briefed scope.

9-2. Under-Documenting Scope

The inverse of over-promising: building a workflow without documenting what it does not do, which leads to users applying it outside its design envelope.

Every deployed AIP Logic workflow must have documented scope boundaries: - What input types it handles - What input types it does not handle (and what happens when it receives them) - What Ontology data it relies on, and what happens when that data is missing or stale - What output format it produces - What the output does not include

Undocumented scope boundaries are discovered by users in the field — often in high-tempo, high-stakes conditions when the workflow is being used in a way it was not designed for. That is not the time for discovery.

9-3. Building Without Defining Correct Output

Before building any AI workflow, define what correct output looks like. Not in vague terms ("a good assessment") but in operational terms ("a paragraph of 100-150 words, flagging all units below 80% on any LOGSTAT class, formatted per the USAREUR-AF LOGSTAT standard, with source Object IDs cited for each flagged unit").

If you cannot define correct output before building, you cannot test the workflow, you cannot evaluate its outputs, and you cannot explain to a reviewer whether what they are seeing is correct. Build the definition of correct first; build the workflow second.

9-4. Deploying Without Human-Review Gates

Any AIP Logic workflow deployed to production without a documented and enforced human review step is in violation of Army CIO AI governance policy and USAREUR-AF C2DAO requirements. This is not a design preference — it is a compliance requirement.

The failure mode to recognize: review gates that exist in documentation but are not enforced in the workflow. A Workshop application that has a "Review" button but allows the output to flow to downstream consumers without pressing it is not a review gate — it is a review suggestion. Enforce the gate in the workflow logic.

9-5. Confusing a Demo With a System

A demo that works once — with curated input data, in a development environment, with an AI engineer walking through it — is not evidence that a workflow works reliably. It is evidence that the workflow works on that input, that day, in that environment.

The gap between "works in demo" and "works in production" is where most AI workflow failures occur. In production: - Input data is messier, less complete, and less well-structured than demo data - Users interact with the workflow in ways the demo did not anticipate - Volume is higher; edge cases appear more frequently - The AI engineer is not present to explain anomalies

Close the gap with: a comprehensive test suite built against operational data samples (not curated demo data), a staged rollout beginning with low-stakes use cases, a monitored production period with explicit anomaly reporting, and a rollback plan that is tested before it is needed.

SUMMARY: THE AI ENGINEER'S DESIGN CHECKLIST

Before submitting any AIP Logic workflow for C2DAO review, confirm:

DRAFT

Item	Confirmed
Use case decomposed through five-question framework (Section 4)	
Human review step defined: who, what, how, enforcement mechanism (Section 6)	
All factual claims grounded in retrieved Ontology data (Section 3)	
Prompt tested against nominal, edge, adversarial, and off-task inputs (Section 7)	
Failure modes documented	
Agent step limit and loop-detection configured (if applicable) (Section 5)	
Output distinguishes retrieved facts from LLM-generated assessments (Section 2)	
Scope boundaries documented (Section 9)	
Rollback procedure defined and tested	
C2DAO authorization obtained before production deployment (Section 8)	

TRANSITION TO TM-40B

This Concepts Guide establishes the mental models you will apply throughout TM-40B. TM-40B task instruction assumes this conceptual foundation — it will not re-explain why RAG grounding matters, why human review gates are enforced in workflow logic, or why prompts require version control. Those questions are answered here.

When TM-40B directs you to configure an AIP Logic workflow, you will understand what you are configuring and why the design choices matter. When TM-40B presents a human review gate, you will understand that it is a doctrinal requirement, not an optional feature. When TM-40B presents a test procedure for a prompt, you will understand what you are testing for.

Proceed to TM-40B, Chapter 1, when you can answer the following without referencing this guide:

1. What is the difference between what an AI engineer builds and what an ML engineer builds on MSS?
2. Why does grounding LLM output in Ontology data reduce hallucination risk?
3. What five questions must be answered before developing an AI use case?
4. What is the difference between meaningful human review and rubber-stamping?
5. Why is a prompt treated as system configuration, not casual instruction?

If any of these questions requires you to look up the answer, re-read the relevant section before proceeding.

DRAFT

CONCEPTS GUIDE — TM-40B COMPANION, AI ENGINEER, MAVEN SMART SYSTEM (MSS)
Headquarters, United States Army Europe and Africa, Wiesbaden, Germany, 2026 Distribution restriction: Approved for public release; distribution is unlimited.

DRAFT